UPTEC XX XXXXX
Degree project 30 credits
April, 2026

Heterogeneous Multi-Robot Collaboration Using Large Language Models

Exploring different sizes and architectures

Rohit Joseph Mamutil

Master's Programme in Embedded Systems

Heterogeneous Multi-Robot Collaboration Using Large Language Models

Rohit Joseph Mamutil

Abstract

Heterogeneous warehouse fleets typically include robots with different roles (e.g., carrier vehicles and loading agents) that must coordinate their movement, timing, and energy usage to sustain high throughput. Designing controllers for such multi-robot systems is challenging due to large discrete action spaces, partial observability, sparse rewards, and safety constraints such as battery depletion.

This thesis investigates whether off-the-shelf large language models (LLMs) can be used as high-level decision-makers for heterogeneous robot collaboration in a task-assignment warehouse environment with battery dynamics. The approach converts the current multi-agent state into a compact natural-language description, queries an LLM to choose an action for each agent, and enforces feasibility by validating proposed actions against environment-defined valid action masks. Different model sizes, planning architectures, and prompt designs are compared in the Battery-TA-RWARE environment.

The current work contributes (i) an end-to-end evaluation pipeline for LLM-driven multi-agent control, (ii) prompt and validation patterns tailored to discrete warehouse actions, and (iii) preliminary experimental results showing that prompt structure, planning architecture, and safeguard logic strongly influence whether LLM-based control remains productive under changing objectives.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Xuezhi Niu Subject reader: Didem Gürdür Broo

Examiner: Ramanathan Thinniyam Srinivasan

Sammanfattning

I heterogena robotflottor i lager används ofta olika robottyper (t.ex. bärande automated guided vehicles (AGVs) och lastningsagenter) som måste samordna sina handlingar för att uppnå hög genomströmning samtidigt som säkerhetskrav, såsom batterinivåer, uppfylls. Att konstruera styrsystem för sådana multi-robot systems (MRSs) är svårt på grund av stora diskreta handlingsrymder, glesa belöningssignaler och behovet av robust koordination.

Detta examensarbete undersöker om färdigtränade LLMs kan användas som beslutsfattare på hög nivå för samarbete mellan heterogena robotar i en lager-miljö med uppgiftsallokering och batteridynamik (Battery-TA-RWARE). Metoden sammanfattar det multi-agentiska tillståndet i naturligt språk, låter en LLM välja en handling per agent och validerar sedan handlingarna mot miljöns tillåtna handlingsmasker.

Arbetet levererar en första utvärderingspipeline för LLM-styrning av multi-agentisk lagerlogistik samt preliminära resultat som visar att en enkel språkbaserad LLM-policy presterar avsevärt sämre än en domänheuristik, vilket motiverar fortsatt arbete med bättre struktur, grounding och koordination.

Acknowledgement I would like to thank my supervisor, Xuezhi Niu, for continuous guidance and constructive feedback throughout this thesis work. I also thank my reviewer, Didem Gürdür Broo, and my examiner, Ramanathan Thinniyam Srinivasan, for their input on the direction and quality of the work. Finally, I am grateful to the developers and maintainers of the open-source environments and tools used in this project.

Contents

Acronyms **vii**

1 Introduction **1**

1.1 Research Questions 2

1.2 Contributions 3

1.3 Report Outline 3

2 Problem Formulation and Objectives **3**

2.1 Problem Formulation 3

2.2 Purpose 4

2.3 Objectives 4

2.4 Delimitations 4

2.5 Ethical and Sustainability Considerations 5

3 Background and Related Work **5**

3.1 Multi-Robot Systems and Coordination 5

3.2 Warehouse Task Assignment Environments 6

3.3 LLM-Based Planning and Grounding 7

3.4 Why Study Model Size and Prompt Design? 7

4 System and Method **8**

4.1 Overview 8

4.2 Environment: Battery-TA-RWARE 8

4.3 LLM Control Loop 9

4.4 Experiment Families and Objectives 10

4.5 Planning Architectures 11
4.6 Prompt Structures 12
4.7 Comparison Axes 13
4.8 Evaluation Metrics 13
4.9 Scenario and Model Matrix 14
4.10 Experimental Protocol (Current Stage) 14

5 Preliminary Results and Discussion 15

5.1 Result Scope and Objective Progression 15
5.2 Objective 1: Shelf-Delivery Experiments 16
5.2.1 Planning architecture 16
5.2.2 Prompt representation 17
5.3 Objective 2: Call-Aware Shelf-Delivery Experiments 17
5.3.1 Prompt representation 18
5.3.2 Adaptation to the changed objective 18
5.3.3 Model size 19
5.3.4 Call-efficiency outcome 20
5.4 Objective 3: Battery-Aware Objective 20
5.5 System-Level Lessons from the Experiments 20
5.6 Limitations of Interpretation 21

6 Conclusions and Future Work 22

6.1 Future Work 23

A Appendix 26

A.1 Reproducibility: Running the Controllers 26

A.2 Prompt Excerpt (Illustrative) 27
A.3 Metric Definitions 27

Acronyms

AGV automated guided vehicle. ii, 2, 6, 8, 9, 12, 14, 18, 20

JSON JavaScript Object Notation. 11, 12, 15, 17, 22

LLM large language model. i, ii, v, 1–4, 6–16, 18, 19, 21–23, 27

MARL multi-agent reinforcement learning. 1, 3, 23

MAS multi-agent system. 5, 6

MRS multi-robot system. ii, 1, 5

PPO proximal policy optimization. 13, 23

RL reinforcement learning. 4, 13

TA-RWARE task-assignment robotic warehouse environment. 8

1 Introduction

In studies of human communication, language use has been framed as a form of joint action in which participants coordinate their individual contributions toward shared outcomes [5]. Accounts of language origins likewise emphasize the role of cooperative motives and shared intentionality in enabling coordination and collaboration [14]. Language has also been linked to social bonding and group-level coordination in evolutionary accounts of human sociality [6].

Motivated by these perspectives, this thesis adopts the working hypothesis that language-capable models with general reasoning ability can serve as high-level decision-makers in multi-agent settings, thereby supporting collaboration and coordination without task-specific training for every new configuration.

To evaluate this hypothesis in a concrete and measurable way, the thesis uses a simplified but structured multi-agent testbed. When autonomous agent fleets operate in a shared environment, high throughput typically requires more than path planning: agents must decide *which* tasks to execute, *when* to coordinate with other roles, and *when* to allocate time to maintenance actions such as recharging. These decisions are particularly challenging in heterogeneous teams where agents have different capabilities and constraints.

Research on MRSs spans architectures, communication, and coordination strategies. Early surveys emphasized fundamental axes such as group architecture, resource conflicts, and learning [4]. Later overviews categorized multi-robot research topics including communication, task allocation/control, and motion coordination [2]. In practical logistics domains, these themes manifest as repeated, interdependent choices: allocate requests to carriers, ensure loaders and carriers meet at shelves, resolve conflicts in shared corridors, and maintain energy safety.

Many approaches address such problems using optimization, hand-engineered heuristics, or multi-agent reinforcement learning (MARL). Optimization and heuristics can be strong and efficient, but they often require careful domain-specific design. MARL can learn sophisticated joint behaviour, but training is expensive, sensitive to reward shaping and partial observability, and may generalize poorly when the environment parameters change. Benchmarking studies highlight that algorithm comparisons are highly task- and metric-dependent, motivating careful protocol design and reporting [13]. Recent work in heterogeneous multi-robot settings also introduces bio-inspired *symbiosis* framings and reward shaping strategies, often combined with centralized training and decentralized execution, to stabilize learning and improve cooperative outcomes [11, 12].

Recent LLMs offer a complementary direction: instead of learning a policy from scratch

for each environment, an LLM can be prompted with a structured description of the current state and asked to output a high-level decision. In robotics, LLMs have been used for instruction following and planning by combining language reasoning with explicit grounding and feasibility checks; for example, by selecting from validated affordances rather than unconstrained free-form actions [1]. However, applying LLMs to multi-agent control introduces additional challenges: actions must remain valid under environment constraints, decisions must be coordinated across agents, and long-horizon objectives must be approximated under limited context and latency budgets.

This thesis studies LLM-driven decision-making in a heterogeneous multi-robot warehouse domain with explicit battery dynamics. The experimental platform is Battery-TARWARE, an adaptation of the Task-Assignment Robotic Warehouse environment [9], featuring two agent types: carrier AGVs and picker agents that must meet at shelf locations for load/unload operations. At each decision point, the current global state is summarized into a compact textual prompt, the LLM proposes an action id per agent, and a validation layer enforces action feasibility via action masks.

1.1 Research Questions

The thesis is guided by the following main research question:

> **Main research question:** How can LLMs be used in heterogeneous multi-robot systems to improve collaboration?

To support a systematic investigation of the main research question, the study considers two sub-questions:

1. **RQ1 (Model size):** How do LLMs of different sizes influence the quality and effectiveness of collaboration in heterogeneous multi-robot systems?
2. **RQ2 (Integration architecture):** Which integration architecture, centralized or decentralized, enables the most effective use of LLMs for coordinating heterogeneous multi-robot teams?

In this thesis, architectural choices include how environment state is translated into language, how proposed decisions are mapped back to executable actions, whether decision-making is performed per-agent or jointly, and how feasibility constraints are imposed (e.g., by restricting/validating candidate actions and using environment feedback for replanning).

1.2 Contributions

The current thesis work contributes:

- An end-to-end pipeline for running and evaluating LLM-driven heterogeneous multi-robot control in Battery-TA-RWARE, including state-to-language encoding, action-id grounding, output parsing, and feasibility validation.
- An experimental setup for comparing models of different sizes and for evaluating centralized versus decentralized LLM integration patterns.
- Preliminary results and an error analysis comparing different LLM planning architectures, prompt forms, and objective formulations.

1.3 Report Outline

Section 2 states the problem formulation, objectives, and delimitations. Section 3 provides background on multi-robot coordination, warehouse task assignment, and LLM-based planning. Section 4 describes the environment and the proposed LLM control loop. Section 5 presents preliminary experiments and discusses observed failure modes. Finally, Section 6 concludes and outlines future work.

2 Problem Formulation and Objectives

2.1 Problem Formulation

Warehouse logistics in heterogeneous fleets can be framed as a continuous decision-making problem: at each time step, each agent must choose a discrete target (shelf, goal, or charging station), subject to dynamic constraints (valid targets, role restrictions, agent busy states) and safety constraints (battery depletion). The objective is to maximize delivery throughput while maintaining robust coordination between roles (carrier–picker interactions) and avoiding failure modes such as deadlocks, clashes, and out-of-charge immobilization.

This thesis investigates LLM-based control as a language-driven alternative for high-level multi-agent decision-making in a constrained warehouse setting. The central question is whether natural-language reasoning can be turned into *actionable* multi-agent decisions once combined with explicit grounding and validation.

2.2 Purpose

The purpose of this thesis is to evaluate the feasibility and limitations of using off-the-shelf LLMs as high-level controllers for heterogeneous multi-robot collaboration in a warehouse task-assignment setting with battery management.

2.3 Objectives

The concrete objectives are:

1. Implement an end-to-end LLM control loop for Battery-TA-RWARE, including prompt generation, output parsing, and feasibility validation.
2. Define an evaluation protocol and metrics that capture throughput, coordination quality, and safety.
3. Compare different LLM sizes, planning architectures, and prompt designs across the experiment objectives studied in this thesis.
4. Identify and characterize failure modes (invalid actions, poor coordination, battery mismanagement) and propose mitigation strategies.

2.4 Delimitations

To keep the study focused and measurable, the following delimitations apply in the current stage:

- The work is evaluated in a simulated, discrete-time environment (Battery-TA-RWARE) rather than on physical robots.
- The LLM is used for high-level target selection; low-level navigation is handled by the environment (A* path planning and collision handling).
- The initial LLM policies are *not* fine-tuned; they are prompted as general-purpose models accessed through a local inference server.
- The reported experiments cover several warehouse sizes and role ratios, but they are still based on fixed seeds and a limited number of experimental sessions.
- The battery-oriented objective family had not yet produced completed aggregate results at the time of writing.

2.5 Ethical and Sustainability Considerations

Although this work is simulation-based, it touches on two practical considerations:

- **Safety:** Policies that ignore constraints (e.g., battery safety) can lead to immobilized agents or unsafe behaviour in real systems. The proposed validation layer is motivated by this gap between natural-language output and executable actions.
- **Energy and compute:** Larger models increase inference cost and energy consumption. A key motivation for comparing different model sizes is to understand whether smaller models can be made effective through better prompting, structure, and validation.

3 Background and Related Work

3.1 Multi-Robot Systems and Coordination

MRSs study how multiple embodied agents share resources, divide work, communicate, and avoid conflicts while pursuing a common objective. The term multi-agent system (MAS) is broader and can include purely software agents, whereas MRS typically emphasizes physical robots operating under motion, sensing, and actuation constraints. Classical overviews of cooperative robotics highlight recurring concerns such as group architectures, resource conflicts, and learning [4]. Survey treatments vary in terminology and scope (e.g., differences in how MAS and MRS are framed, and how communication and coordination taxonomies are defined) [16, 2]. For broader multi-agent coordination perspectives, conference proceedings such as AAMAS provide additional representative examples, including coordinated MARL methods that explicitly trade off coordination benefits against communication bandwidth [18]. In heterogeneous teams, these concerns intensify because roles and capabilities differ, and success often requires explicit synchronization (e.g., meeting at the same physical location at the same time).

In this thesis, *collaboration* refers to interdependent task execution among heterogeneous roles, where agents must coordinate to complete an operation (e.g., a load/unload interaction). *Coordination* refers to mechanisms that prevent conflicts and deadlocks (e.g., collision avoidance and congestion handling), and *cooperation* refers to pursuing a shared objective even when agents may act more independently. These terms are not used consistently across communities; some MAS discussions historically emphasize distributed stationary nodes, whereas modern multi-agent learning often includes mobile, embodied agents. To reduce ambiguity, the thesis primarily uses MRS when

referring to embodied robots and uses MAS to denote the broader class of multi-agent systems.

Coordination strategies can be described as *static* (offline, convention-based) or *dynamic* (online, reactive), depending on whether coordination rules are established prior to execution or synthesized during execution from sensed/communicated information [16]. Communication can likewise be categorized as *implicit* (information inferred via sensing or environment traces) versus *explicit* (direct message passing), trading off scalability against accuracy and bandwidth [16].

Planning is a central lens for collaboration: multi-robot planning is often discussed as task planning (who does what) and motion planning (how to move), with optimal formulations typically being NP-hard and thus motivating approximate methods and heuristics [16]. In particular, multi-robot task planning includes task decomposition (splitting a mission into subtasks) and task allocation (assigning subtasks to agents), with much of the literature focusing on allocation due to tractability and benchmarking convenience [16]. This observation aligns with the thesis hypothesis: LLMs may be useful as a structured decision module for decomposition and allocation when paired with explicit grounding, feasibility constraints, and environment feedback.

3.2 Warehouse Task Assignment Environments

Warehouse domains provide a structured but challenging testbed for multi-agent decision-making. Tasks are naturally decomposed into repeated cycles: select a requested shelf, bring it to a delivery station, return it to storage, and repeat. When multiple agents operate in the same grid, the action space and coordination requirements scale quickly with environment size and fleet size.

This thesis uses Battery-TA-RWARE, an adaptation of the Task-Assignment Robotic Warehouse environment introduced in [9]. The environment models two heterogeneous agent types:

- **AGVs:** carrier agents that transport shelves between storage locations and goal stations.
- **Pickers:** support agents that must meet AGVs at shelf locations for loading and unloading.

Battery-TA-RWARE extends the task-assignment setting with battery depletion and charging stations, forcing agents to balance throughput with energy safety. Low-level

movement is handled by environment dynamics (including A* path planning), while the controller selects discrete targets representing shelves, goals, or charging stations.

3.3 LLM-Based Planning and Grounding

LLMs can produce coherent natural-language plans, but robot control requires converting language into *grounded*, *feasible* actions. One influential pattern is to constrain the LLM to select among validated, environment-defined options. For example, SayCan combines language-level reasoning with an affordance model to filter actions that are executable in the current state [1]. Other work frames LLMs as planners that can propose action sequences given a symbolic interface to the environment, often coupled with explicit checking and replanning [7]. In language-only settings, prompting patterns such as interleaving reasoning and actions have been studied to improve decision quality under tool or environment feedback [17]. Recent multi-robot systems work explores *multi-agent* prompting where each robot maintains a separate dialog context and coordination emerges through structured communication plus environment feedback (e.g., collision checking and in-context plan revision) [10]. Recent work argues that closed-loop state feedback and a separation between high-level planning and low-level control can reduce hallucinated actions and improve long-horizon goal satisfaction in robot task planning with LLMs [3]. In multi-robot task planning contexts, prior thesis work also emphasizes the need for a symbolic abstraction layer that translates sensor/state representations into language-level predicates that LLMs can reliably consume [8].

These ideas motivate the design in this thesis: use a prompt that provides a compact but explicit mapping from environment state to action identifiers, require outputs to be parseable into discrete actions, and validate them against the environment's current action constraints.

3.4 Why Study Model Size and Prompt Design?

In applied robotics and embedded contexts, compute and latency budgets may favour smaller models. However, smaller models may require more structured prompting, tighter candidate sets, and stronger validation to reach acceptable performance. Recent robotics work explicitly motivates prompt design as a lever for compensating limited model capacity and improving decision quality in constrained deployments [15, 10]. By evaluating multiple model sizes and prompt variants under the same environment and metrics, the thesis aims to clarify which components (model capacity, prompt structure, validation) most strongly affect feasibility, coordination quality, and throughput.

4 System and Method

4.1 Overview

The thesis uses an empirical, experimental methodology:

1. Implement an LLM-driven control loop for a simulated heterogeneous warehouse environment.
2. Define a set of baselines and evaluation metrics.
3. Run controlled experiments (fixed environment ids and seeds) across multiple LLM models, planning architectures, and prompt variants.
4. Analyze throughput, safety, and coordination outcomes as well as qualitative failure modes.

4.2 Environment: Battery-TA-RWARE

Battery-TA-RWARE is a discrete-time, grid-based warehouse simulation derived from task-assignment robotic warehouse environment (TA-RWARE) [9]. At any time, a set of shelves is requested. Carrier AGVs must transport requested shelves to goal locations and then return them to storage. Picker agents are required to meet AGVs at shelf locations for load and unload interactions, creating explicit heterogeneous coordination constraints. Agents additionally have a battery state that decreases with movement and interactions; charging stations are available at fixed locations.

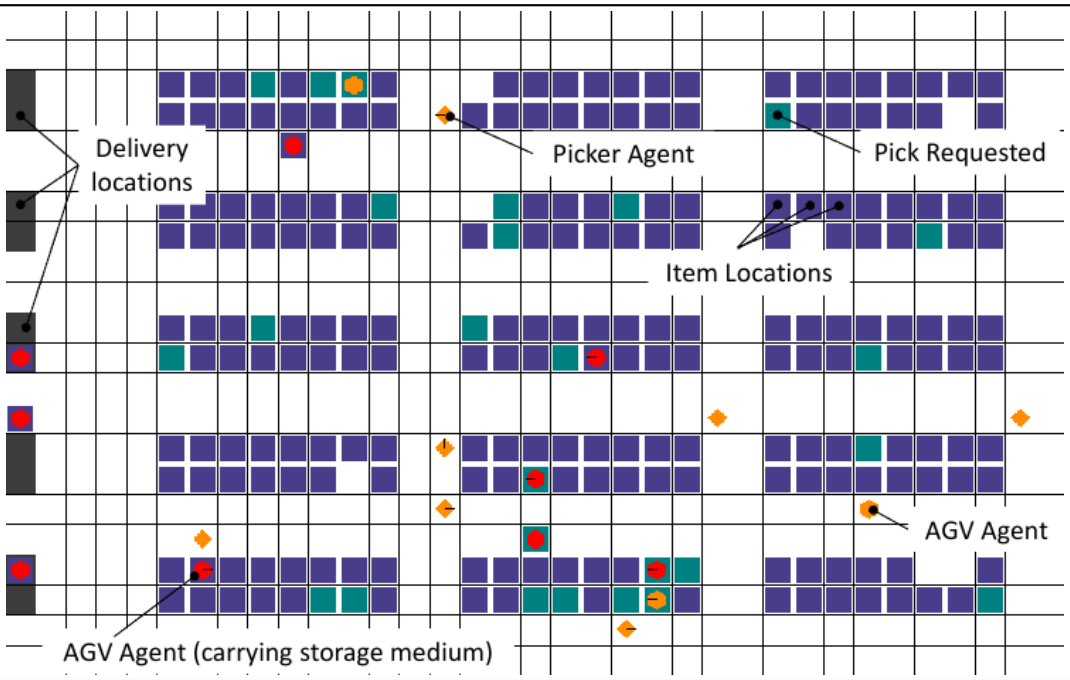

Figure 1 Illustration of the task-assignment warehouse setting used in this thesis. Two heterogeneous roles must coordinate: carrier AGVs transport shelves, while picker agents support loading and unloading at shelf locations. Battery dynamics add a safety constraint that can force agents to recharge.

The environment exposes a discrete action space where each action corresponds to selecting a target location:

- **Shelf actions:** choose a shelf location (e.g., to pick up or return a shelf).
- **Goal actions:** choose a delivery station (primarily relevant when an AGV carries a requested shelf).
- **Charging actions:** choose a charging station to recover battery.

Low-level motion toward the selected target is handled internally by the environment (A* path planning and collision resolution), while the controller is evaluated on the quality of these high-level decisions.

4.3 LLM Control Loop

The LLM policy is implemented as a text-in / discrete-action-out controller:

1. **State summarization:** extract salient features of the global state (requested shelves and their action ids, per-agent position/target/battery/carrying status, charging occupancy).
2. **Candidate action set:** compute a valid-action mask from the environment; optionally restrict the candidate list shown to the LLM to reduce prompt size.
3. **Prompting:** generate a per-agent prompt describing the agent role and the current state, including explicit mappings from requested objects to action ids.
4. **Querying:** send the prompt to a local inference server (Ollama) and obtain a text response.
5. **Parsing:** parse an integer action id (and optionally an action hold duration) from the response.
6. **Validation:** validate the parsed action id against the current valid-action mask; invalid outputs are discarded and trigger replanning.
7. **Execution:** apply the validated joint action in the environment for one step (or hold the action for multiple steps, depending on configuration).

Two design choices are central for feasibility:

- **Grounding via action ids:** the LLM does not output coordinates or free-form text commands; instead it selects from discrete action ids that are already defined by the environment.
- **Hard validation:** action masks enforce constraints such as invalid goals for picker agents and busy-state restrictions. Invalid LLM outputs are never executed.

4.4 Experiment Families and Objectives

The implemented experiments are grouped into three objective families.

Objective 1: maximize total shelf deliveries. The first family focuses on raw throughput. These experiments are stored under results/shelf and were designed to answer two questions: whether a centralized or shared-context planning architecture is more effective, and whether natural-language or JavaScript Object Notation (JSON)-style prompting is the more suitable representation for that objective. The family was implemented in two architectures:

- **Centralized planner:** scripts/run_centralized_llm_experiments.py. One LLM prompt receives the global state and returns a joint plan for all non-busy agents.
- **Shared-context decentralized planner:** scripts/run_shared_context_llm_expt.py. Each agent is planned separately, but each prompt still contains a shared global summary of requests, charging occupancy, and waiting support relations.

For both architectures, two prompt styles were evaluated:

- **Natural-language form:** the prompt and response use a compact textual format.
- **JSON form:** the prompt and response use a more structured object-and-array representation.

Objective 2: optimize LLM calls while maintaining delivery throughput. The second family is implemented in scripts/run_obj2_shared_context_llm.py and stored under results/calls. This family keeps the shared-context, per-agent architecture, but changes the objective from delivery-only to a joint objective: increase shelf deliveries while reducing unnecessary LLM queries. In practice, the runner supports action holding and busy-state reuse so that the model is not queried at every environment step. This family was used primarily to study the practical trade-off between throughput, model size, and call efficiency, and more broadly to test whether the LLM-based control framework remained useful when the objective changed.

Objective 3: battery-aware objective. A third family was planned for explicitly battery-oriented evaluation and corresponding result storage under results/battery. At the time of writing, this experimental branch had not yet produced completed result files, so it is described as planned work rather than evaluated work.

4.5 Planning Architectures

The experiments compare two distinct ways of integrating the LLM into the decision loop.

Centralized planning. In the centralized architecture, one LLM call plans all agents that currently need a decision. This design exposes the full global state and all relevant candidate actions in one prompt. Its intended advantage is explicit joint coordination. Its

main risk is prompt overload: as the number of agents and candidate actions increases, the prompt becomes large and more difficult to parse reliably.

Shared-context per-agent planning. In the shared-context architecture, the LLM plans one agent at a time. Each prompt still includes global information, but the final decision concerns only the current agent. This reduces the output burden and simplifies parsing, at the cost of requiring coordination to emerge from repeated local decisions rather than from one explicit joint plan. In the thesis evaluation, this architecture is the main vehicle for comparing prompt forms and model sizes under a common decision structure.

The run_obj2_shared_context_llm.py variant extends the shared-context architecture with explicit action-hold durations and logging for call-efficiency analysis. It therefore remains decentralized in prompt structure while adding stronger runner-side control over when replanning is allowed.

4.6 Prompt Structures

The prompt comparison in this thesis is not limited to natural language versus JSON. The prompt design varies along two axes:

- **Planning scope:** centralized joint planning versus per-agent planning with shared context.
- **Output structure:** natural-language responses versus JSON responses.

Across the different scripts, the prompts evolved to include:

- explicit requested-shelf to action-id mappings,
- role-specific rules for AGVs and pickers,
- support relations such as waiting-for-picker context,
- battery-state abstractions,
- candidate action filtering or prioritization,
- and, in Objective 2, an explicit Steps field intended to reduce unnecessary LLM calls.

The purpose of this progression was experimental rather than aesthetic: to determine which combination of planning scope and prompt structure yields valid, productive macro actions under the constraints of the warehouse task. In particular, the prompt comparison was intended to separate two questions: whether planning should be centralized or per-agent, and whether the same decision content is handled better in natural-language form or in a more structured JSON form.

4.7 Comparison Axes

The current report focuses on internal comparisons within the LLM-based control framework rather than on comparison against an external baseline. The main comparison axes are:

- model size,
- planning architecture (centralized versus shared-context per-agent),
- prompt form (natural language versus JSON),
- and objective formulation (shelf-delivery only, call-aware shelf delivery, and planned battery-aware control).

4.8 Evaluation Metrics

The main metric is delivery throughput, expressed as a pick rate (deliveries per hour) derived from the number of delivered shelves and the simulated episode duration. Additional metrics are collected to capture coordination and safety:

- total shelf deliveries,
- number of clashes and deadlocks/stuck events,
- distance travelled by each agent type,
- episode return (environment reward),
- episode length and inference speed/latency (when measured).

4.9 Scenario and Model Matrix

The same scenario catalogue is reused across the experiment families:

- tiny_balanced_1v1,
- small_balanced_2v2,
- medium_balanced_4v4,
- small_agv_heavy_4v2,
- medium_agv_heavy_6v2,
- small_picker_heavy_2v4.

These scenarios were selected to probe different coordination pressures: simple balanced collaboration, increased concurrency, larger warehouses, AGV-heavy support bottlenecks, and picker-heavy underutilization.

The model set spans smaller and larger local models:

- llama3.2:1b,
- llama3.2:3b,
- mistral:7b,
- gemma3:12b,
- phi4,
- qwen2.5:14b.

4.10 Experimental Protocol (Current Stage)

To support reproducibility during prompt iteration, the reported experiment sessions were run with fixed environment ids and a fixed seed. This choice makes behavioural debugging easier, but it also limits statistical confidence. The current thesis therefore treats the results as experimental evidence about architecture and prompt behaviour, not as final performance claims.

The progression of the experimental campaign was:

1. Objective 1 shelf-delivery experiments, comparing centralized and shared-context planning and comparing natural-language and JSON prompt forms,
2. Objective 2 call-aware shelf-delivery experiments, using a later shared-context controller with stronger prompt structure and safeguards,
3. Objective 3 battery-aware experiments, planned but not yet completed.

One broader aim of this progression was to test whether the same general LLM-based control framework could adapt as the stated objective changed. Section 5 therefore first analyzes Objective 1 as the shelf-delivery baseline, then Objective 2 as a changed objective using a later and more structured shared-context controller, and finally notes Objective 3 as planned future evaluation. In the current simulation setup, a run can achieve at most 20 deliveries, and the results chapter uses that ceiling to contextualize reported scores.

5 Preliminary Results and Discussion

5.1 Result Scope and Objective Progression

This chapter analyzes the completed experiment families stored under results/shelf and results/calls. The experimental program was organized around three objectives. Objective 1 focused on maximizing total shelf deliveries. Objective 2 retained shelf delivery as the task outcome, but changed the optimization target to improving delivery while reducing unnecessary LLM calls. Objective 3 was planned as an explicitly battery-aware objective, but that branch had not yet produced completed aggregate results at the time of writing.

One broader goal of the thesis was to test whether an LLM-based control framework could remain useful when the objective changed. In other words, the question was not only which configuration performed best, but also whether the same overall control paradigm could move from pure throughput optimization to a joint throughput-and-call-efficiency objective, and later toward a battery-oriented objective.

For interpreting the reported numbers, one additional fact is important: in the current simulation setup, 20 deliveries is the maximum achievable total for a run. Reported delivery counts should therefore be read relative to this ceiling. A run with only a few deliveries is not merely weak in absolute terms; it is far below what the environment allows when coordination succeeds.

Objective 2 should also not be read as a direct continuation of Objective 1 under an unchanged controller. It used a later shared-context runner with stronger prompt design, action holding, and additional safeguard logic. The Objective 2 results therefore reflect both a changed objective and a more mature controller.

5.2 Objective 1: Shelf-Delivery Experiments

Objective 1 provides the main basis for comparing planning architecture and prompt form under the simplest thesis objective: maximize total shelf deliveries. These experiments are stored under results/shelf and reflect an earlier stage of the system, before the later call-aware shared-context runner was introduced.

5.2.1 Planning architecture

The architecture comparison in Objective 1 is summarized in Table 1.

Experiment family	Runs	Total deliveries	Average deliveries/run
Centralized + natural	36	15	0.42
Centralized + JSON	36	21	0.58
Shared-context + natural	36	25	0.69
Shared-context + JSON	36	0	0.00

Table 1 Aggregate outcome of the Objective 1 shelf-delivery experiments across models and scenarios.

The overall pattern suggests that shared-context planning was more promising than centralized planning, particularly when paired with natural-language prompting. Shared-context plus natural language produced the highest aggregate delivery count in this family. The likely reason is that per-agent planning reduced the burden of each query while still exposing the global state needed for coordination. In the centralized architecture, one prompt had to describe the full state and return multiple coordinated actions at once, which created a heavier joint decision problem for the tested models.

At the same time, the Objective 1 results should not be overstated. Even the best configuration in this family remained far below the 20-delivery per-run ceiling. This means Objective 1 is more useful as an architecture comparison than as evidence of strong final performance. It suggests that shared-context planning was the more promising direction, but it does not demonstrate that the Objective 1 controller had already become effective in absolute terms.

5.2.2 Prompt representation

Objective 1 is also the main basis for comparing prompt representation under the shelf-only objective. In this family, natural-language prompting performed better than JSON prompting in the shared-context architecture. The recorded shared-context natural-language runs produced 25 deliveries across 36 runs, whereas the recorded shared-context JSON runs produced zero. The centralized comparison was less one-sided, but all centralized variants remained weak overall and still far below the per-run delivery ceiling.

The correct interpretation is cautious. The early JSON prompts were not fully comparable to the natural-language prompts. They used deeper nesting and exposed some decision-critical state less directly. Later prompt revisions made the JSON form much closer to the natural prompt in information content and ordering. For that reason, the early Objective 1 JSON results should not be taken as proof that structured prompting is inherently unsuitable. They do, however, show that in the earlier shelf-delivery setup, natural language was clearly the more effective prompt form.

Taken together, Objective 1 answers two thesis questions in a preliminary but useful way: shared-context planning looked more promising than centralized planning, and natural-language prompting looked more effective than JSON prompting in the earlier shelf-delivery controller. Both findings should be read as informative rather than definitive because Objective 1 belonged to an earlier stage of the system.

5.3 Objective 2: Call-Aware Shelf-Delivery Experiments

Objective 2 changed the goal from pure delivery throughput to a joint objective: preserve delivery performance while reducing unnecessary LLM calls. These experiments are stored under results/calls and use a later shared-context runner with stronger prompt structure, action holding, and additional safeguard logic. For that reason, Objective 2 is the main basis for evaluating model size and for asking whether the LLM-based framework remained useful under a changed objective.

Across 36 runs, the completed Objective 2 natural-language shared-context session produced 303 deliveries, corresponding to an average of 8.42 deliveries per run. This was a substantial improvement over the Objective 1 aggregate results, although that improvement cannot be attributed to the objective change alone because the controller itself also became more mature.

5.3.1 Prompt representation

The completed Objective 2 sessions provide a direct prompt-format comparison within the later shared-context runner family. The natural-language session produced 303 deliveries across 36 runs, corresponding to 8.42 deliveries per run. The completed JSON session produced 53 deliveries across the same number of runs, corresponding to 1.47 deliveries per run. Efficiency measures show the same pattern: the natural-language session required approximately 204.7 LLM calls per delivery and 469.5 seconds per delivery, while the JSON session required approximately 1078.4 calls per delivery and 3057.4 seconds per delivery.

The later Objective 2 JSON session was not a complete failure, since it produced non-zero throughput, mainly through gemma3:12b and phi4. However, it remained substantially weaker than the natural-language version of the same runner family. The natural-language session had 2 zero-delivery runs out of 36, whereas the JSON session had 26 zero-delivery runs out of 36. The completed Objective 2 results therefore support the same overall conclusion as Objective 1: natural-language prompting remained more effective than JSON prompting in the evaluated setup, even after the later prompt and runner refinements.

5.3.2 Adaptation to the changed objective

Objective 2 provides partial evidence that the framework remained useful when the optimization target changed from throughput only to throughput plus call efficiency. The later shared-context controller still produced substantial delivery throughput while explicitly trying to reduce unnecessary calls. In several scenarios, the best-performing model reached or approached the 20-delivery ceiling, which indicates that productive control remained possible under the revised objective.

This result is not a clean transfer study, because Objective 2 was not evaluated under a frozen Objective 1 controller. Runner logic, prompt design, and safeguards changed together with the stated objective. The completed experiments therefore support a narrower conclusion: the evolving LLM-based framework remained viable under the changed objective, but the available evidence does not isolate objective adaptation from the effect of the later controller improvements.

5.3.3 Model size

The clearest model-size comparison in the thesis comes from Objective 2 and is summarized in Table 2.

Model	Total deliveries	Average deliveries/scenario	Seconds/delivery
llama3.2:3b	73	12.17	238.5
gemma3:12b	63	10.50	460.2
mistral:7b	59	9.83	530.5
qwen2.5:14b	46	7.67	495.6
llama3.2:1b	36	6.00	371.4
phi4	26	4.33	1091.6

Table 2 Aggregate results for the Objective 2 shared-context natural-language session.

Table 2 suggests that model size mattered, but not in a monotonic way. Larger models did not consistently achieve better throughput, nor did they consistently justify their additional latency. The strongest practical operating point in this session was llama3.2:3b, which combined the highest total deliveries with the lowest seconds-per-delivery among the tested models. This is notable because it is neither the smallest nor the largest model in the set.

The model-size effect was also scenario-dependent. Some scenarios appear to have rewarded richer context interpretation, while others rewarded faster and more consistent local decisions. This means the thesis does not support a universal best-size claim. It does support the narrower conclusion that larger models were not uniformly better, and that a mid-sized local model offered the strongest practical trade-off in the most complete Objective 2 session.

The completed Objective 2 JSON session also adds an important nuance to the model-size picture. In that session, only gemma3:12b and phi4 produced substantial throughput, with 26 and 25 deliveries respectively, while both llama3.2 variants produced zero deliveries across all six scenarios. This suggests that structured prompting in the later call-aware setup placed a heavier burden on the smaller models than the corresponding natural-language prompts did. Even so, the overall prompt-format effect remained larger than the model-size effect: the best JSON models still remained well below the best natural-language results.

5.3.4 Call-efficiency outcome

Objective 2 should be judged on both throughput and LLM usage. Raw calls-per-delivery were approximately 146.2 for phi4, 158.3 for gemma3:12b, 167.0 for qwen2.5:14b, 167.7 for llama3.2:3b, 287.0 for mistral:7b, and 316.3 for llama3.2:1b. This metric should still be interpreted with care, since a model can appear efficient simply by doing too little. For that reason, the most convincing result is not the lowest raw call ratio, but the balance between delivery throughput and call usage. By that criterion, llama3.2:3b again appears to be the most practical choice in the completed Objective 2 session.

Objective 2 therefore appears to have made meaningful progress toward the call-aware goal, but only partially. Action holding and reuse reduced some unnecessary queries, yet many query resolutions still arose from runner-side intervention rather than from stable long-horizon plans. The result is best described as a workable but still imperfect call-aware controller rather than a fully solved low-query policy.

5.4 Objective 3: Battery-Aware Objective

Objective 3 was planned to test whether the same general framework could adapt again to a further changed objective centered on battery behaviour. The intended result family is stored under results/battery. At the time of writing, however, this branch had not yet produced completed aggregate results. For that reason, Objective 3 remains future work rather than an analyzable result set.

This means the thesis can only partially answer the broader adaptation question. There is evidence that the framework remained useful when moving from Objective 1 to Objective 2, but there is not yet completed evidence for adaptation to an explicitly battery-oriented objective.

5.5 System-Level Lessons from the Experiments

The strongest overall lesson from Objectives 1 and 2 is that LLMs were most useful when bounded by rule-based logic rather than treated as unconstrained controllers. The experiments do not provide positive evidence that giving full responsibility to the LLM improved outcomes. Instead, stronger behaviour appeared only when the model operated inside a grounded and safeguarded decision loop with candidate shaping, action validation, and conflict-handling logic.

Several observations support this interpretation. First, Objective 1 remained weak across

all architecture and prompt combinations, with most configurations finishing far below the 20-delivery ceiling. Second, the stronger Objective 2 results appeared only in a later shared-context controller that relied on additional structure such as action holding and stronger control logic around when and how decisions were reused. Third, the JSON experiments showed that valid syntax alone was not sufficient: parseable outputs still often led to poor decisions.

The failure patterns point in the same direction. In the Objective 2 natural-language shared-context session, 32 of 36 runs ended with an exact 500-step no-delivery tail. These poor runs often stopped far below the 20-delivery ceiling and were usually not associated with strong explicit conflict or stuck signals. They were more consistent with macro-level coordination failures such as stale waiting loops, support-allocation problems, or charging oscillation. This indicates that higher-level coordination did not become reliable simply because an LLM was placed in the loop.

The architectural lesson is therefore narrow but important. In the present system, the LLM contributed most when it selected among already grounded and validated macro actions. Correctness-critical functions such as action validity, exclusivity, and conflict handling remained better suited to explicit rule-based logic. The experiments therefore support a constrained view of LLM control rather than a fully free-form one.

5.6 Limitations of Interpretation

Several limitations should be kept in mind when interpreting these results.

- The reported sessions use fixed seeds and should therefore be treated as behavioural evidence rather than as statistically robust rankings.
- The experiment families were not clean ablations. Prompt design, runner logic, and some environment-side behaviour evolved across the objectives, which limits causal attribution.
- Objective 2 should not be read as a pure apples-to-apples continuation of Objective 1, since both the objective and the controller changed.
- Objective 3 had not yet produced completed results, so battery-aware adaptation remains open.
- Some outcomes were scenario-specific and may not persist under broader reruns.

Taken together, the results support four cautious conclusions. Objective 1 established that shared-context planning was more promising than centralized planning and that

natural-language prompting was more effective than JSON prompting in the earlier shelf-delivery controller. Objective 2 showed that the framework could remain useful under a changed, call-aware objective, especially in a more structured shared-context setup. Within that stronger family, a mid-sized model offered the best practical trade-off. Most importantly, the experiments suggest that LLM control was most viable when bounded by rule-based safeguards rather than treated as a replacement for them.

6 Conclusions and Future Work

This thesis explores LLM-driven decision-making for heterogeneous multi-robot collaboration in a task-assignment warehouse setting with battery constraints. The work implemented an end-to-end control loop that (i) grounds decisions to discrete environment action identifiers, (ii) queries an LLM using centralized or per-agent shared-context prompts, and (iii) enforces feasibility through action-mask validation.

The results should be interpreted relative to a 20-delivery per-run ceiling in the current simulation setup. Against that ceiling, Objective 1 established the initial shelf-delivery comparison between architectures and prompt forms. In that earlier controller family, shared-context planning was more promising than centralized planning, and natural-language prompting was more effective than JSON prompting. These results were informative, but they remained far below the achievable ceiling and therefore served more as an indication of architectural direction than as evidence of strong final performance.

Objective 2 then changed the optimization target from pure shelf delivery to call-aware shelf delivery while also using a later, safer shared-context runner. The completed Objective 2 sessions show that the framework remained usable under the changed objective, especially in the more structured shared-context setup. Within that family, model size mattered, but not monotonically: larger models were not uniformly better, and the strongest practical trade-off in the natural-language session came from a mid-sized model, `llama3.2:3b`, rather than from the largest available model. The completed Objective 2 JSON session produced non-zero throughput, but it remained substantially weaker than natural language in both throughput and call efficiency.

Objective 3, the battery-aware objective, remains open because completed aggregate result files were not yet available. This means the thesis only partially answers the broader question of whether the framework adapts smoothly as the objective changes. The evidence supports viability across the shift from Objective 1 to Objective 2, but not yet for a dedicated battery-centered objective.

The strongest overall lesson is not that the LLM replaced controller logic, but that it con-

tributed best as a constrained macro-action selector inside a safeguarded system. Many poor runs ended in long non-productive tails, often far below the 20-delivery ceiling and usually without clear low-level collision deadlocks. This suggests that higher-level coordination did not become reliable simply by inserting an LLM into the loop. Action validation, candidate shaping, conflict handling, and other rule-based logic remained necessary to keep the system stable and productive.

The overall evidence in this thesis therefore supports a constrained view of LLM control. In the evaluated warehouse setting, LLMs were most useful inside a strongly structured decision loop, while correctness-critical behaviour remained better handled by explicit rule-based logic.

6.1 Future Work

- **Structured outputs and tighter parsing:** require the model to output a strict schema (e.g., {`action_id, hold_steps, rationale`}) and reject non-conforming responses to reduce ambiguity.
- **Centralized versus decentralized planning:** compare per-agent prompts (current approach) with a centralized planner that outputs a joint action vector, including explicit coordination constraints.
- **Candidate-set design:** learn or compute compact candidate action sets (e.g., nearest requested shelves, relevant support targets for pickers) while guaranteeing that necessary actions remain available.
- **Multi-seed and scaling studies:** extend evaluation to many random seeds and larger layouts (more shelves, more agents) to quantify robustness and scalability.
- **MARL baselines:** train/evaluate proximal policy optimization (PPO) or other MARL methods on Battery-TA-RWARE and compare to LLM in throughput, generalization, and compute cost.
- **Cost-aware deployment:** measure inference latency, token usage, and energy cost across model sizes to assess feasibility for embedded or real-time settings.

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[2] T. Arai, E. Pagello, and L. E. Parker, "Editorial: Advances in multi-robot systems," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 655–661, 2002.

[3] V. Bhat, A. U. Kaypak, P. Krishnamurthy, R. Karri, and F. Khorrami, "Grounding large language models for robot task planning using closed-loop state feedback," 2025, received: 27 April 2025; revised: 10 October 2025; accepted: 17 October 2025.

[4] Y. U. Cao, A. S. Fukunaga, and A. B. Kahng, "Cooperative mobile robotics: Antecedents and directions," *Autonomous Robots*, vol. 4, no. 1, pp. 7–27, 1997.

[5] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.

[6] R. Dunbar, *Grooming, Gossip, and the Evolution of Language*. Cambridge, MA, USA: Harvard University Press, 1996.

[7] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *arXiv preprint arXiv:2201.07207*, 2022.

[8] Y. Huang, "Collaborative multi-robot task planning using large language models," Master's thesis, Uppsala University, 2025, accessed 2026-02-26. [Online]. Available: https://uu.diva-portal.org/smash/get/diva2:2023514/FULLTEXT01.pdf

[9] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. Wing Keung To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Börsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," 2022.

[10] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics*

and Automation (ICRA), 2024, pp. 286–299, accessed 2026-02-26. [Online]. Available: https://ieeexplore.ieee.org/document/10610855

[11] X. Niu, N. C. Barajas, and D. G. Broo, "Enabling symbiosis in multi-robot systems through multi-agent reinforcement learning," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–7.

[12] X. Niu and D. G. Broo, "Investigating symbiosis in robotic ecosystems: A case study for multi-robot reinforcement learning reward shaping," in *2025 9th International Conference on Robotics and Automation Sciences (ICRAS)*, 2025, pp. 112–117.

[13] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," 2020, accessed 2026-02-26. [Online]. Available: https://arxiv.org/abs/2006.07869

[14] M. Tomasello, *Origins of Human Communication*. Cambridge, MA, USA: MIT Press, 2008.

[15] L. Xiao and T. Yamasaki, "Probing prompt design for socially compliant robot navigation with vision-language models," 2026, accessed 2026-02-26.

[16] Z. Yan, N. Jouandeau, and A. A. Cherif, "A survey and analysis of multi-robot coordination," *International Journal of Advanced Robotic Systems*, vol. 10, no. 12, 2013, accessed 2026-02-26. [Online]. Available: https://journals.sagepub.com/doi/full/10.5772/57313

[17] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[18] C. Zhang and V. Lesser, "Coordinating multi-agent reinforcement learning with limited communication," in *Proceedings of the 12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2013, accessed 2026-02-26. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2013/docs/p1101.pdf

A Appendix

A.1 Reproducibility: Running the Controllers

The experiments in this thesis are executed from the Battery-TA-RWARE repository in this workspace. Example commands:

```
python Battery-TA-RWARE/scripts/run_llm.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1 \
  --model llama3.2:3b
```

Listing 1: Run the basic per-agent LLM controller for one episode.

```
python Battery-TA-RWARE/scripts/run_centralized_llm_experiments.py \
  --prompt_format language \
  --seed 0
```

Listing 2: Run the centralized shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_shared_context_llm_expt.py \
  --prompt_format language \
  --seed 0
```

Listing 3: Run the shared-context shelf-delivery experiments.

```
python Battery-TA-RWARE/scripts/run_obj2_shared_context_llm.py \
  --prompt_format language \
  --max_steps 0 \
  --max_busy_steps_for_replan 10 \
  --max_inactivity_steps 500 \
  --disable_support_needed_soon \
  --find_path_agent_aware_always \
  --seed 0
```

Listing 4: Run the Objective 2 shared-context call-aware experiments.

```
python Battery-TA-RWARE/scripts/run_heuristic.py \
  --env_id tarware-tiny-1agvs-1pickers-partialobs-chg-v1 \
  --seed 0 \
  --num_episodes 1
```

Listing 5: Run the heuristic policy for one episode.

The main result folders discussed in the thesis are:

- results/shelf/central_llm_experiments: Objective 1, centralized planning.
- results/shelf/shared_context_llm_experiments: Objective 1, shared-context per-agent planning.
- results/calls/shared_context: Objective 2, shared-context per-agent planning with call-efficiency objective.
- results/battery: planned battery-oriented experiments (not yet populated with completed aggregate sessions at the time of writing).

A.2 Prompt Excerpt (Illustrative)

The LLM is prompted with an explicit mapping from relevant state elements to discrete action identifiers. The exact prompt depends on the experiment family: centralized prompts describe all agents at once, while shared-context prompts describe one current agent together with selected global context. The full prompts are generated programmatically; the excerpt below illustrates the style (line breaks and ordering may vary):

```
Current global state:
- Requested shelves:
  - Requested shelf 24 is at position (11,5). To go there, choose action id 50.
  - Requested shelf 39 is at position (6,8). To go there, choose action id 29.
...
You are agent_0 (type=AGV).
Choose exactly one action_id from the candidate list.
Output format: action_id=<integer>
```

Listing 6: Illustrative prompt excerpt.

A.3 Metric Definitions

- **Total shelf deliveries:** number of requested shelves delivered to goal locations during an episode.
- **Overall pick rate:** deliveries/hour computed as

$$\text{pick_rate} = \frac{\text{deliveries} \cdot 3600}{5 \cdot \text{episode_steps}},$$

  where 5 steps/second is the environment's time scale used by the logging script.

- **Clashes and stuck events:** environment-defined counts related to collision resolution and deadlock-like behaviour.
- **Global episode return:** sum of environment rewards across all agents and time steps.